

Configuración en Spring Boot

application.properties, yml y perfiles

Java con Spring Boot de 0 a Pro

Playlist: Fundamentos de Spring Boot • Video #5



¿Qué aprenderás hoy?

1 Configuraciones externas en Spring Boot

Cómo manejar parámetros fuera del código fuente

3 Inyección de valores con @Value y @ConfigurationProperties

Dos enfoques para acceder a configuraciones

2 Formatos application.properties y application.yml

Diferencias y cuándo usar cada uno

4 Perfiles para distintos entornos

Configuraciones específicas para dev, test y producción

Agenda del video

01

Configuración externa

Por qué evitar valores hardcodeados

02

Formatos de archivos

Properties vs YAML

03

Inyección de valores

@Value y @ConfigurationProperties

04

Perfiles de entorno

Gestión de múltiples configuraciones

05

Práctica

Ejemplos de configuración real

¿Por qué configuración externa?

La configuración externa permite que nuestras aplicaciones sean flexibles y adaptables sin necesidad de recompilar código. Es una práctica esencial para aplicaciones profesionales.

✗ Hardcode

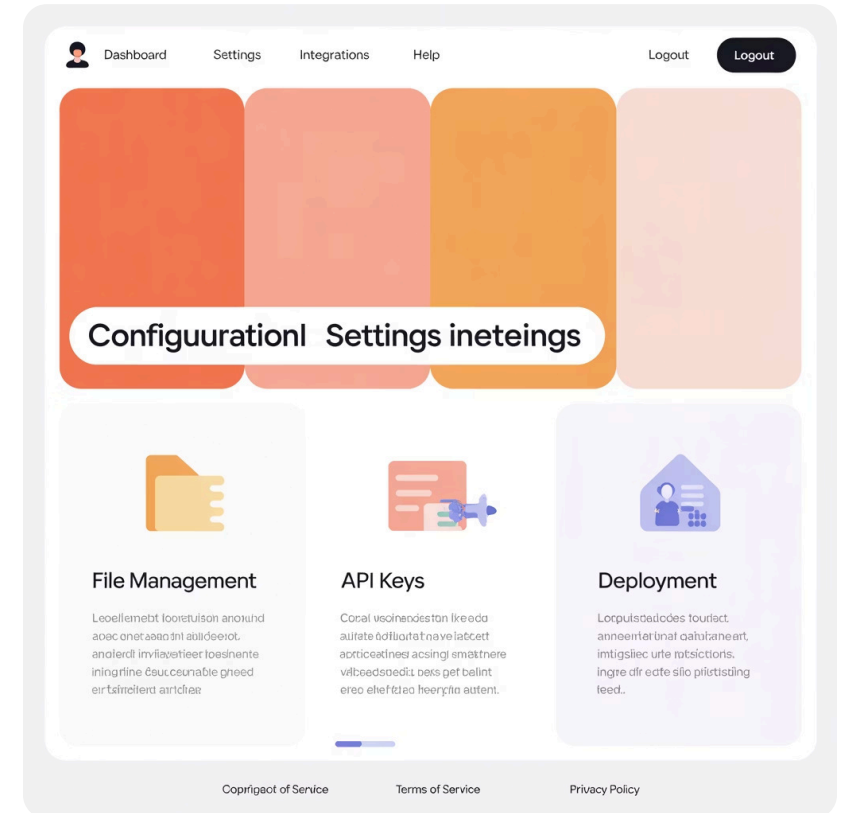
Valores fijos en el código

Requiere recompilación para cambios

✓ Configuración externa

Valores en archivos separados

Cambios sin recompilar





application.properties

El formato tradicional de configuración en Spring Boot utiliza la sintaxis clave=valor. Es simple, directo y ampliamente compatible con herramientas Java.

Sintaxis simple

```
server.port=8080  
spring.application.name=mi-app  
logging.level.com.ejemplo=DEBUG
```

Características

- Formato clave=valor
- Una propiedad por línea
- Comentarios con #
- Amplia compatibilidad

application.yml

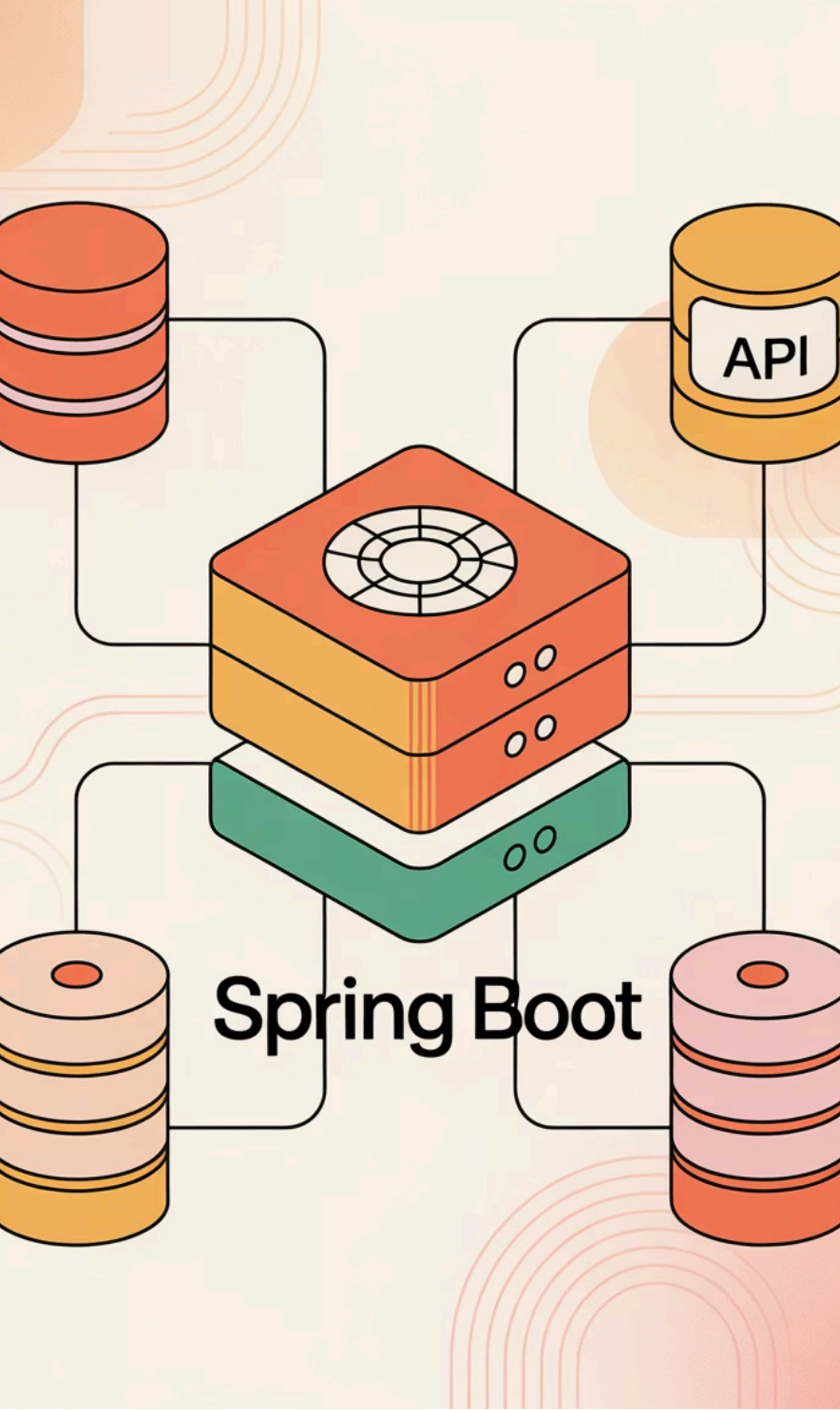
YAML ofrece una sintaxis más legible y estructurada, especialmente útil para configuraciones complejas con múltiples niveles de anidamiento.

Sintaxis estructurada

```
server:
  port: 8080
spring:
  application:
    name: mi-app
  logging:
    level:
      com.ejemplo: DEBUG
```

Ventajas YAML

- Más legible para estructuras complejas
- Menos repetitivo
- Soporte nativo para listas y mapas
- Indentación significativa



Ejemplo: Configuración básica del servidor

Veamos cómo configurar aspectos básicos de nuestra aplicación Spring Boot usando ambos formatos para las mismas propiedades.

application.properties

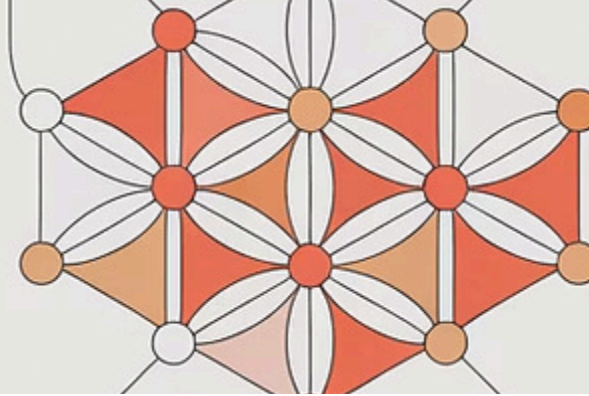
```
# Configuración del servidor
server.port=8090
server.servlet.context-path=/api

# Información de la aplicación
spring.application.name=configurac
ion-demo
management.endpoints.web.expos
ure.include=health,info
```

application.yml

```
# Configuración del servidor
server:
  port: 8090
  servlet:
    context-path: /api

# Información de la aplicación
spring:
  application:
    name: configuracion-demo
  management:
    endpoints:
      web:
        exposure:
          include: health,info
```



Inyección con @Value

La anotación @Value permite inyectar valores específicos desde archivos de configuración directamente en campos, métodos o parámetros de constructores.



Configuración

```
app.nombre=Mi Aplicación  
app.version=1.0.0  
app.timeout=5000
```



Uso en Java

```
@Component  
public class AppService {  
    @Value("${app.nombre}")  
    private String nombre;  
  
    @Value("${app.timeout:3000}")  
    private int timeout;  
}
```

 **Tip:** Usa valores por defecto con `${propiedad:valorDefault}` para mayor robustez.

Mapeo con @ConfigurationProperties

Para grupos de propiedades relacionadas, @ConfigurationProperties ofrece un enfoque más elegante y type-safe que múltiples @Value.

Configuración

```
app.database.url=localhost:5432
app.database.username=admin
app.database.password=secret
app.database.pool.min=5
app.database.pool.max=20
```

Clase Java

```
@ConfigurationProperties(prefix = "app.database")
@Component
public class DatabaseConfig {
    private String url;
    private String username;
    private String password;
    private Pool pool = new Pool();

    // getters y setters

    public static class Pool {
        private int min = 5;
        private int max = 10;
        // getters y setters
    }
}
```

Ventajas

- Type safety y validación
- Agrupación lógica
- Autocompletado en IDE
- Metadatos de configuración

Perfiles de configuración

Los perfiles permiten tener diferentes configuraciones para distintos entornos (desarrollo, testing, producción) manteniendo el mismo código base.



Desarrollo (dev)

Base de datos H2 en memoria, logging detallado, puerto 8080. Configuración optimizada para desarrollo local con reinicio rápido.



Testing (test)

Base de datos de test, logging mínimo, métricas desactivadas. Entorno aislado para pruebas automatizadas.



Producción (prod)

Base de datos real, logging de errores, seguridad activada, puerto 80. Configuración optimizada para rendimiento.

Environment profiles

Development Profile

Testing

Production



Cómo activar un perfil

Spring Boot ofrece múltiples formas de activar perfiles, desde archivos de configuración hasta variables de entorno y argumentos de línea de comandos.



En application.properties

```
spring.profiles.active=dev
```



Variable de entorno

```
export SPRING_PROFILES_ACTIVE=prod
```



Línea de comandos

```
java -jar mi-app.jar --spring.profiles.active=test
```



En IDE

Configuración en propiedades de ejecución o variables de entorno del IDE

 Los archivos específicos de perfil siguen el patrón: application-{perfil}.properties

Beneficios de la configuración externa

Flexibilidad

Cambios de configuración sin recompilar código

- Ajustes en tiempo real
- Adaptación a diferentes entornos
- Configuración por cliente

Seguridad

Credenciales fuera del código fuente

- Secretos en variables de entorno
- Configuración específica por entorno
- Control de acceso granular

Mantenibilidad

Código más limpio y fácil de mantener

- Separación de responsabilidades
- Configuración centralizada
- Testing más sencillo

Configuration Management System



Resumen de lo aprendido

Has adquirido los conocimientos fundamentales para manejar configuraciones en Spring Boot de manera profesional y escalable.

Archivos de configuración

application.properties y application.yml para centralizar configuraciones

Inyección de valores

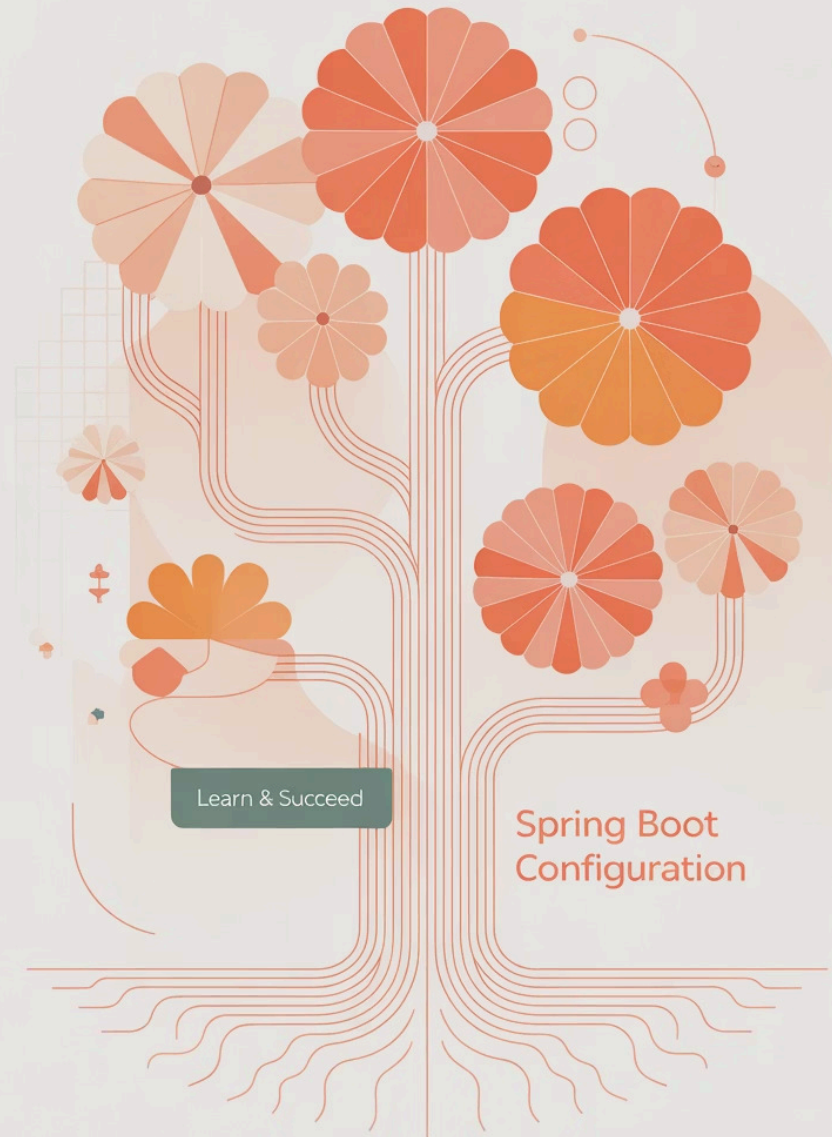
@Value para valores simples y @ConfigurationProperties para grupos

Perfiles de entorno

Configuraciones específicas para dev, test y producción

Activación de perfiles

Múltiples formas de seleccionar el entorno activo



Próximos pasos

Con las bases de configuración dominadas, estás listo para explorar características más avanzadas que harán tu experiencia de desarrollo aún más fluida.



Auto-configuración

Cómo Spring Boot configura automáticamente componentes



Developer Experience

Herramientas que mejoran la productividad



Configuración avanzada

Validación y metadatos de configuración



Siguiente video: Profundizaremos en cómo Spring Boot automatiza la configuración de componentes comunes.



¡Sigue configurando tu éxito!

La configuración externa es la base de aplicaciones robustas y mantenibles

Java con Spring Boot de 0 a Pro

¡Nos vemos en el próximo video!